1차시 학생 활동지 · 놀이 속 알고리즘 찾기

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 문제 발견 · 게임: 업다운 · 가위바위보 · 기억력 게임 체험

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 파이썬 개발 환경에서 .py 파일을 실행할 수 있다.
- ☐ print() 함수로 원하는 문장을 출력할 수 있다.
- ☐ 익숙한 게임의 규칙을 입력–처리–출력(IPO)으로 분석할 수 있다.

핵심 질문

> 우리가 늘 하던 놀이(게임)의 규칙을 컴퓨터는 어떻게 알고 있을까?

나의 생각: _____

활동 1. 게임 체험 기록

게임	몇 번 만에 끝났나?	재미있었던 점 / 아쉬운 점
업다운		
가위바위보		
기억력 게임		

활동 2. 입력–처리–출력 분석표

게임	입력	처리	출력	종료 조건
업다운	1~100 사이 숫자	정답과 비교	UP/DOWN/정답	정답 또는 기회 소진
가위바위보				
기억력 게임				
우리 모둠이 만들고 싶은 게임				

활동 3. 생각 정리

1. 세 게임에 공통으로 쓰인 프로그래밍 요소는? _____

2. 업다운 게임을 가장 빨리 맞히는 나의 전략은? _____

3. 컴퓨터에게 규칙을 알려 줄 때 사람에게 설명할 때와 다른 점은? _____

핵심 개념 빈칸 채우기

1. 알고리즘 = 문제 해결 절차, 프로그램 = (_____) 코드로 표현한 것
2. 모든 (_____) 입력 → 처리 → 출력 구조로 분석할 수 있다.
3. (_____)는 출력, #은 주석, "문자열" * n은 반복

자기 평가

항목	잘함	보통	노력 필요
파이썬 개발 환경에서 .py 파일을 실행할 수 있다.	☐	☐	☐
print() 함수로 원하는 문장을 출력할 수 있다.	☐	☐	☐
익숙한 게임의 규칙을 입력–처리–출력(IPO)으로 분석할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

2차시 학생 활동지 · 게임 데이터 추상화: 변수·자료형·입력

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 문제 구조화 · 게임: 업다운 게임의 데이터 찾기

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 게임에 꼭 필요한 데이터를 골라내는 추상화를 할 수 있다.
- ☐ 변수에 값을 저장하고, 정수·실수·문자열·불 자료형을 구분할 수 있다.
- ☐ input()으로 입력받고 int()로 형 변환하여 계산할 수 있다.

핵심 질문

> 게임을 만들려면 컴퓨터가 무엇을 '기억'하고 있어야 할까?

나의 생각: _____

활동 1. 추상화: 꼭 필요한 것과 빼도 되는 것

꼭 필요한 데이터	변수 이름	자료형	처음 값
정답 숫자	secret	int	1~100 무작위

빼도 되는 것: _____

활동 2. 결과 예상하고 확인하기

코드	예상 결과	실제 결과
`print("3" * 3)`		
`print(3 * 3)`		
`print(7 / 2, 7 // 2, 7 % 2)`		
`print("50" + "1")`		
`print(int("50") + 1)`		

활동 3. 생각 정리

2 ** 7 = 이다. 업다운 게임의 기회가 7번인 이유를 적어 보자.

_____

핵심 개념 빈칸 채우기

1. 추상화: 게임에 꼭 필요한 (_____) 기회, 범위, 기록)만 골라낸다.
2. 변수 = 이름 붙은 상자, (_____)는 대입
3. (_____)은 항상 문자열 → 계산하려면 int()
4. (_____) 몫, % 나머지, ** 거듭제곱

자기 평가

항목	잘함	보통	노력 필요
게임에 꼭 필요한 데이터를 골라내는 추상화를 할 수 있다.	☐	☐	☐
변수에 값을 저장하고, 정수·실수·문자열·불 자료형을 구분할 수 있다.	☐	☐	☐
input()으로 입력받고 int()로 형 변환하여 계산할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

3차시 학생 활동지 · 규칙 분해와 조건문

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 문제 구조화 · 게임: 업다운 판정 · 가위바위보 판정

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 게임 규칙을 작은 하위 문제로 분해하여 분해 트리로 나타낼 수 있다.
- ☐ 비교 연산자와 논리 연산자의 결과(True/False)를 설명할 수 있다.
- ☐ if–elif–else 문으로 여러 경우를 판정하는 프로그램을 작성할 수 있다.

핵심 질문

> 게임 규칙을 어떤 작은 문제로 나누어야 컴퓨터가 판단할 수 있을까?

나의 생각: _____

활동 1. 우리 모둠 게임 분해 트리

```
(게임 이름) <span class="blank"></span>
 ├──── 준비하기 :
 ├──── 한 번 시도하기
 │     ├────
 │     ├────
 │     └────
 └──── 끝내기 :
```

활동 2. 판정 규칙 표

경우	조건식 (파이썬)	출력
정답	`guess == secret`	정답!
더 큰 수		
더 작은 수		

활동 3. 가위바위보 (나 - 컴퓨터) % 3 확인

나	컴퓨터	계산식	값	결과
가위(0)	보(2)	(0-2)%3		

나	컴퓨터	계산식	값	결과
보(2)	바위(1)			
바위(1)	보(2)			

핵심 개념 빈칸 채우기

1. 분해: 게임 = 준비 + (_____) + 종료
2. (_____) 연산의 결과는 True / False
3. (_____) 위에서부터 처음 참인 블록 하나만 실행
4. 콜론 : 과 (_____) 4칸은 필수

자기 평가

항목	잘함	보통	노력 필요
게임 규칙을 작은 하위 문제로 분해하여 분해 트리로 나타낼 수 있다.	☐	☐	☐
비교 연산자와 논리 연산자의 결과(True/False)를 설명할 수 있다.	☐	☐	☐
if–elif–else 문으로 여러 경우를 판정하는 프로그램을 작성할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

4차시 학생 활동지 · 순서도와 while 반복문: 업다운 완성

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 컴퓨팅 설계 · 게임: 업다운 게임 (기본 버전)

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 게임 한 판의 흐름을 순서도 또는 의사코드로 설계할 수 있다.
- ☐ while 문과 break, continue를 이용해 반복과 종료 조건을 구현할 수 있다.
- ☐ 업다운 게임 전체 코드를 읽고 각 부분의 역할을 설명할 수 있다.

핵심 질문

> 같은 일을 '정답을 맞힐 때까지' 반복하려면 어떻게 설계해야 할까?

나의 생각: _____

활동 1. 의사코드 완성하기

```
정답 ← <span class="blank"></span>, 기회 ← <span class="blank"></span>
<span class="blank"></span> 동안 반복:
    입력 ← 사용자 입력
    만약 입력 = 정답: <span class="blank"></span> 후 반복 종료
    아니고 입력 < 정답: <span class="blank"></span>
    아니면: <span class="blank"></span>
    기회 ← <span class="blank"></span>
반복이 끝까지 돌았으면: <span class="blank"></span>
```

활동 2. 순서도 그리기

아래 칸에 업다운 게임 한 판의 순서도를 그리세요. (시작/끝, 입력, 판단, 처리, 흐름선 기호 사용)

활동 3. 이진 탐색 실험

범위	이진 탐색 최대 횟수	근거 (2의 거듭제곱)
1~100	7	$2^7 = 128 \geq 100$
1~1000		
1~10000		

핵심 개념 빈칸 채우기

1. 코딩 전에 (_____) 반복과 종료 조건을 먼저 설계
2. while 조건: 참인 동안 반복 / break 탈출 / (_____) 건너뛰기
3. (_____) ~ else: break 없이 끝났을 때만 else
4. 가운데 값 고르기 = 이진 탐색 → (_____) 최대 7번

자기 평가

항목	잘 함	보통	노력 필요
게임 한 판의 흐름을 순서도 또는 의사코드로 설계할 수 있다.	☐	☐	☐
while 문과 break, continue를 이용해 반복과 종료 조건을 구현할 수 있다.	☐	☐	☐
업다운 게임 전체 코드를 읽고 각 부분의 역할을 설명할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

5차시 학생 활동지 · 리스트와 random: 가위바위보

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 컴퓨팅 설계 · 게임: 가위바위보 (기본 버전)

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 리스트를 만들고 인덱스로 값을 읽고 추가할 수 있다.
- ☐ random 모듈로 무작위 선택을 구현할 수 있다.
- ☐ in 연산자와 나머지 연산으로 입력 검사와 승패 판정을 할 수 있다.

핵심 질문

> 여러 개의 값을 하나의 이름으로 다루려면 어떤 자료구조가 필요할까?

나의 생각: _____

활동 1. 리스트 연습

`items = ["검", "방패", "포션"]` 일 때

코드	결과
`items[0]`	
`items[-1]`	
`len(items)`	
`items.index("방패")`	
`"활" in items`	

활동 2. 규칙 바꾸기 설계

'먼저 3승' 규칙으로 바꿀 때 고쳐야 할 줄:

1. _____
2. _____

5가지 가위바위보(가위·바위·보·도마뱀·스팍)를 `%` 5로 판정할 수 있을까? 이유는?

_____

핵심 개념 빈칸 채우기

1. 리스트: 여러 값을 (_____) 인덱스는 0부터
2. (_____) 추가 · index 위치 · in 포함 여부 · len 개수
3. (_____) / choice / sample
4. 순환 규칙 → (_____) 연산 % 으로 표현

자기 평가

항목	잘함	보통	노력 필요
리스트를 만들고 인덱스로 값을 읽고 추가할 수 있다.	☐	☐	☐
random 모듈로 무작위 선택을 구현할 수 있다.	☐	☐	☐
in 연산자와 나머지 연산으로 입력 검사와 승패 판정을 할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

6차시 학생 활동지 · for 반복문과 문자열: 숫자 야구

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 컴퓨팅 설계 · 게임: 숫자 야구 (기본 버전)

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ for 문과 range()로 정해진 횟수만큼 반복할 수 있다.
- ☐ 문자열과 리스트의 인덱싱·슬라이싱을 활용할 수 있다.
- ☐ 숫자 야구의 스트라이크·볼 판정 알고리즘을 설계하고 구현할 수 있다.

핵심 질문

> 세 자리를 하나씩 비교하려면 반복을 어떻게 설계해야 할까?

나의 생각: _____

활동 1. 추적표 완성하기

정답 `['2','9','4']`, 추측 `['9','2','4']`

i	guess[i]	answer[i]	판정	strike	ball
0					
1					
2					

활동 2. 개선 설계

'서로 다른 숫자'가 아닌 입력(112)을 막는 조건식을 적어 보자.

힌트: `len(set(guess))`는 서로 다른 글자의 개수

`if <span class="blank"></span>:`

핵심 개념 빈칸 채우기

1. for 변수 in (_____) 정해진 횟수·모든 원소 반복
2. (_____) 슬라이싱 모두 끝 번호는 미포함
3. list("175") ↔ (_____)
4. 변수 변화를 표로 따라가기 = (_____)

자기 평가

항목	잘함	보통	노력 필요
for 문과 range()로 정해진 횟수만큼 반복할 수 있다.	☐	☐	☐
문자열과 리스트의 인덱싱·슬라이싱을 활용할 수 있다.	☐	☐	☐
숫자 야구의 스트라이크·볼 판정 알고리즘을 설계하고 구현할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

7차시 학생 활동지 · 리스트 누적·비교와 문자열 메서드: 기억력 게임

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 컴퓨팅 설계 · 게임: 기억력 게임 (기본 버전)

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 반복할 때마다 리스트에 값을 누적하고, 리스트끼리 비교할 수 있다.
- ☐ strip, replace, split, lower, find 등 문자열 메서드를 목적에 맞게 사용할 수 있다.
- ☐ 무한 반복(while True)과 break로 종료 시점을 제어할 수 있다.

핵심 질문

> 사용자의 입력을 '공정하게' 비교하려면 무엇을 정리해야 할까?

나의 생각: _____

활동 1. 문자열 메서드 결과 예상

코드	예상	실제
`" 바위 ".strip()`		
`"1 2 3".replace(" ", "")`		
`"a,b,c".split(",")`		
`"PyThOn".lower()`		
`"game.py".find(".")`		

활동 2. 기억력 게임 개선 아이디어

문제점	개선 아이디어	필요한 코드/메서드
스크롤하면 숫자가 보임		
앞뒤 공백 입력 시 오답		

핵심 개념 빈칸 채우기

1. while True + (_____) 종료 시점을 반복 안에서 결정
2. 리스트 == 비교는 (_____) 모두 같아야 True

3. strip · (_____) · split · lower · isdigit · find
4. (_____) 새 값을 돌려줌 → 다시 저장

자기 평가

항목	잘 함	보통	노력 필요
반복할 때마다 리스트에 값을 누적하고, 리스트끼리 비교할 수 있다.	☐	☐	☐
strip, replace, split, lower, find 등 문자열 메서드를 목적에 맞게 사용할 수 있다.	☐	☐	☐
무한 반복(while True)과 break로 종료 시점을 제어할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

8차시 학생 활동지 · 함수로 나누기: 업다운·기억력 게임 함수 버전

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 업다운 (함수 버전) · 기억력 게임 (함수 버전)

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ def로 함수를 정의하고 호출할 수 있다.
- ☐ 매개변수와 반환값(return)의 역할을 설명할 수 있다.
- ☐ 기존 코드를 기능별 함수와 main() 구조로 리팩터링할 수 있다.

핵심 질문

> 분해한 '작은 문제'를 코드에서도 따로따로 다루려면 어떻게 해야 할까?

나의 생각: _____

활동 1. 함수 설계표 (우리 모둠 게임)

함수 이름	매개변수	반환값	하는 일
main	없음	없음	전체 흐름

활동 2. 호출 결과 예상

```
def judge(guess, secret):
    if guess == secret:
        return "correct"
    return "up" if guess < secret else "down"
```

호출	결과
`judge(10, 10)`	
`judge(3, 70)`	
`judge(99, 1)`	

핵심 개념 빈칸 채우기

1. 함수 = 하위 문제 (_____) 이름 붙인 코드 묶음
2. 매개변수로 받고, (_____) 돌려준다
3. 기본값 (_____) 난이도 같은 옵션을 쉽게
4. main() + if **name** == (_____) 구조

자기 평가

항목	잘함	보통	노력 필요
def로 함수를 정의하고 호출할 수 있다.	☐	☐	☐
매개변수와 반환값(return)의 역할을 설명할 수 있다.	☐	☐	☐
기존 코드를 기능별 함수와 main() 구조로 리팩터링할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

9차시 학생 활동지 · 딕셔너리: 규칙을 데이터로 표현하기

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 가위바위보 (함수 버전)

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 딕셔너리를 만들고 키로 값을 읽고 수정할 수 있다.
- ☐ 게임 규칙과 점수를 딕셔너리로 표현하여 조건문을 단순하게 만들 수 있다.
- ☐ 딕셔너리의 keys, values, items를 반복문과 함께 활용할 수 있다.

핵심 질문

> '무엇이 무엇을 이긴다'는 규칙을 if 문 대신 데이터로 적을 수 있을까?

나의 생각: _____

활동 1. 리스트? 딕셔너리?

게임 데이터	리스트 / 딕셔너리	이유
입력한 숫자 기록		
가위바위보 승패 규칙		
플레이어별 점수		
영단어와 뜻		

활동 2. 5가지 가위바위보 규칙 모델링

각 선택이 이기는 상대 2개를 리스트로 적어 보자.

```
BEATS = {
    "가위": ["보", "도마뱀"],
    "바위": [ <span class="blank"></span> , <span class="blank"></span> ],
    "보": [ <span class="blank"></span> , <span class="blank"></span> ],
    "도마뱀": [ <span class="blank"></span> , <span class="blank"></span> ],
    "스팍": [ <span class="blank"></span> , <span class="blank"></span> ],
}
```

판정식: `"win" if cpu <span class="blank"></span> BEATS[me] else "lose"`

핵심 개념 빈칸 채우기

1. (_____) {키: 값} → 이름으로 찾기
2. (_____) 데이터로
3. 없는 키는 (_____) → in 검사 또는 get()
4. (_____) 키와 값을 함께 반복

자기 평가

항목	잘함	보통	노력 필요
딕셔너리를 만들고 키로 값을 읽고 수정할 수 있다.	☐	☐	☐
게임 규칙과 점수를 딕셔너리로 표현하여 조건문을 단순하게 만들 수 있다.	☐	☐	☐
딕셔너리의 keys, values, items를 반복문과 함께 활용할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

10차시 학생 활동지 · 2차원 리스트와 중첩 반복: 판정표와 기록표

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 가위바위보 판정표 버전

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 2차원 리스트를 만들고 [행][열] 인덱싱으로 값을 읽을 수 있다.
- ☐ 중첩 반복문으로 표 형태의 데이터를 처리하고 출력할 수 있다.
- ☐ 게임 규칙(판정표)과 기록(기록표)을 2차원 리스트로 모델링할 수 있다.

핵심 질문

> 표(행과 열)로 된 규칙과 기록은 어떤 자료구조에 담아야 할까?

나의 생각: _____

활동 1. 판정표 직접 채우기

값: 0 비김, 1 이김, 2 짐 (행 = 나, 열 = 컴퓨터)

나 \ 컴퓨터	가위(0)	바위(1)	보(2)
가위(0)			
바위(1)			
보(2)			

활동 2. 우리 모둠 게임의 2차원 데이터

우리 게임에서 표로 저장할 수 있는 것은?

데이터 이름	행의 의미	열의 의미	예시 한 줄

핵심 개념 빈칸 채우기

1. (_____) 리스트 = 리스트 안의 리스트 = 표
2. (_____) — 순서 주의!
3. 중첩 (_____) 바깥 = 행, 안쪽 = 칸

4. (_____) 표로 모델링

자기 평가

항목	잘함	보통	노력 필요
2차원 리스트를 만들고 [행][열] 인덱싱으로 값을 읽을 수 있다.	☐	☐	☐
중첩 반복문으로 표 형태의 데이터를 처리하고 출력할 수 있다.	☐	☐	☐
게임 규칙(판정표)과 기록(기록표)을 2차원 리스트로 모델링할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

11차시 학생 활동지 · 종합 실습: 영단어 행맨

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 영단어 행맨 (난이도 중)

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 딕셔너리·리스트·함수·반복·조건을 종합하여 게임을 구현할 수 있다.
- ☐ 제너레이터 표현식과 all() 함수로 '모두 만족하는지'를 검사할 수 있다.
- ☐ 기존 게임에 새 기능을 설계하고 추가할 수 있다.

핵심 질문

> 지금까지 배운 도구를 모두 모으면 어떤 게임을 만들 수 있을까?

나의 생각: _____

활동 1. 코드 지도 그리기

줄/함수	하는 일	사용한 개념
`WORDS`		
`show()`		
`ask_letter()`		
`play()`		

활동 2. 새 기능 설계

추가할 기능: _____

필요한 데이터(변수/리스트/딕셔너리): _____

고칠 함수와 위치: _____

의사코드:

핵심 개념 빈칸 채우기

1. 행맨 = (_____) + 리스트 + 함수 + 반복 + 조건의 종합

2. (_____) 표현식으로 보드를 한 줄에
3. (_____) 모두 참 / any() 하나라도 참
4. True = 1, (_____) = 0

자기 평가

항목	잘함	보통	노력 필요
딕셔너리·리스트·함수·반복·조건을 종합하여 게임을 구현할 수 있다.	☐	☐	☐
제너레이터 표현식과 all() 함수로 '모두 만족하는지'를 검사할 수 있다.	☐	☐	☐
기존 게임에 새 기능을 설계하고 추가할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

12차시 학생 활동지 · 심화: 던전 탈출과 BFS 탐색

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 던전 탈출 (난이도 상)

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 2차원 격자에서 방향 딕셔너리와 튜플로 이동을 구현할 수 있다.
- ☐ BFS(너비 우선 탐색)의 원리를 설명하고 최단 거리 문제에 적용할 수 있다.
- ☐ dataclass와 try–except의 쓰임을 이해할 수 있다.

핵심 질문

> 컴퓨터는 미로에서 출구로 가는 길이 있는지 어떻게 알아낼까?

나의 생각: _____

활동 1. 손으로 BFS 해 보기

출발 S에서 각 칸까지의 최단 거리를 적으세요. (#은 벽)

S(0)		#	
	#		
			#
#			E

E까지의 최단 거리: _____

활동 2. 던전 개조 설계

추가하고 싶은 칸 종류(예: 함정 X, 열쇠 K): _____

밟았을 때 일어나는 일: _____

고쳐야 할 코드(문자 목록, 가중치, if 문): _____

핵심 개념 빈칸 채우기

1. 격자 이동 = 위치 튜플 + 방향 (_____)
2. (_____) 큐로 가까운 칸부터 → 최단 거리
3. (_____) 방문 여부 확인

4. (_____) → 다음 차시 클래스로 연결

자기 평가

항목	잘함	보통	노력 필요
2차원 격자에서 방향 딕셔너리와 튜플로 이동을 구현할 수 있다.	☐	☐	☐
BFS(너비 우선 탐색)의 원리를 설명하고 최단 거리 문제에 적용할 수 있다.	☐	☐	☐
dataclass와 try-except의 쓰임을 이해할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

13차시 학생 활동지 · 클래스와 객체: 붕어빵 틀로 게임 요소 만들기

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 핑퐁 게임 미리 보기 · Paddle 클래스 읽기

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 클래스와 객체, 속성과 메서드의 관계를 설명할 수 있다.
- ☐ __init__과 self를 사용하여 클래스를 정의하고 객체를 여러 개 만들 수 있다.
- ☐ 핑퐁 게임을 구성하는 객체(패들, 공, 점수판, 게임)를 찾아 설계할 수 있다.

핵심 질문

> 패들 2개, 공 1개, 점수판 1개… 비슷한 것을 여러 개 만들 때 코드를 어떻게 묶어야 할까?

나의 생각: _____

활동 1. 핑퐁 게임 객체 찾기

클래스	속성 (가진 정보)	메서드 (하는 일)	몇 개?
Paddle	rect, color, keys, speed	handle_input, draw	2
Ball			
Scoreboard			
Game			

활동 2. 상수 바꾸기 실험

상수	원래 값	바꾼 값	게임의 변화
PADDLE_HEIGHT	90		
BALL_SPEED	6		
WIN_SCORE	5		
FPS	60		

핵심 개념 빈칸 채우기

1. 클래스 = (_____) 객체 = 실물

2. (_____) + 메서드(함수)
3. (_____) 생성 시 자동 실행, self: 호출한 그 객체
4. 핑퐁 = Game이 Paddle×2, Ball, (_____) 지휘

자기 평가

항목	잘 함	보통	노력 필요
클래스와 객체, 속성과 메서드의 관계를 설명할 수 있다.	☐	☐	☐
__init__과 self를 사용하여 클래스를 정의하고 객체를 여러 개 만들 수 있다.	☐	☐	☐
핑퐁 게임을 구성하는 객체(패들, 공, 점수판, 게임)를 찾아 설계할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

14차시 학생 활동지 · pygame 게임 루프: 입력 → 업데이트 → 그리기

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 핑퐁 1단계 (step1_game_loop.py) · main.py · game.py

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ pygame을 설치하고 창을 띄우는 프로그램을 실행할 수 있다.
- ☐ 게임 루프의 4단계(입력·업데이트·그리기·시간 조절)를 설명할 수 있다.
- ☐ 화면 좌표계와 FPS의 의미를 이해하고 그리기 함수를 사용할 수 있다.

핵심 질문

> 그래픽 게임은 어떻게 '멈추지 않고' 계속 움직이는 것처럼 보일까?

나의 생각: _____

활동 1. 게임 루프 분해하기

단계	step1 코드	Game 클래스 메서드
(1) 입력	`for event in pygame.event.get():`	
(2) 업데이트		
(3) 그리기		
(4) 시간 조절		

활동 2. 좌표 계산 연습

화면 800 × 500, 패들 14 × 90, 여백 30

질문	계산식	답
화면 가운데 x		
패들이 세로 가운데에 오는 y		
오른쪽 패들의 x		
속도 7, 60FPS일 때 1초 이동 거리		

핵심 개념 빈칸 채우기

1. 게임 루프: 입력 → (_____) → 그리기 → 시간 조절
2. 화면 좌표: 왼쪽 위 (_____) 아래로 y 증가
3. (_____) × FPS = 1초 이동 거리
4. (_____) = 루프, main.py = 시작 버튼

자기 평가

항목	잘함	보통	노력 필요
pygame을 설치하고 창을 띄우는 프로그램을 실행할 수 있다.	☐	☐	☐
게임 루프의 4단계(입력·업데이트·그리기·시간 조절)를 설명할 수 있다.	☐	☐	☐
화면 좌표계와 FPS의 의미를 이해하고 그리기 함수를 사용할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

15차시 학생 활동지 · Paddle 클래스: 키 입력과 경계 처리

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 핑퐁 2단계 (step2_paddle_class.py) · paddle.py

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 이벤트 방식 입력과 키 상태 방식 입력의 차이를 설명할 수 있다.
- ☐ 딕셔너리로 조작 키를 전달하여 하나의 클래스로 서로 다른 패들을 만들 수 있다.
- ☐ 경계 처리(clamp)로 객체가 화면 밖으로 나가지 않게 할 수 있다.

핵심 질문

> 같은 Paddle 클래스인데 어떻게 왼쪽은 W/S, 오른쪽은 ↑/↓로 움직일까?

나의 생각: _____

활동 1. 입력 방식 분류

기능	이벤트 / 키 상태	이유
패들 위로 이동		
R 재시작		
ESC 종료		
(내 아이디어) 스페이스로 일시정지		

활동 2. handle_input 추적

H=500, PH=90, speed=7, 처음 top=10

프레임	눌린 키	이동 후 top	경계 처리 후 top
1	위		
2	위		
3	아래		

핵심 개념 빈칸 채우기

1. (_____) = 누른 순간 한 번 / get_pressed = 누르는 동안 계속

2. keys (_____) 객체마다 다른 조작 키
3. 경계 처리(clamp): rect.top ≥ 0, (_____) ≤ 화면 높이
4. 좌표 계산: 가운데 = (_____) - 크기/2

자기 평가

항목	잘 함	보통	노력 필요
이벤트 방식 입력과 키 상태 방식 입력의 차이를 설명할 수 있다.	☐	☐	☐
딕셔너리로 조작 키를 전달하여 하나의 클래스로 서로 다른 패들을 만들 수 있다.	☐	☐	☐
경계 처리(clamp)로 객체가 화면 밖으로 나가지 않게 할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

16차시 학생 활동지 · Ball 클래스: 속도 벡터·반사·충돌

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 핑퐁 3단계 (step3_ball_bounce.py) · ball.py

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 속도 벡터(vx, vy)로 물체의 등속 운동을 구현할 수 있다.
- ☐ 반사의 원리를 속도 부호 변경으로 모델링할 수 있다.
- ☐ 직사각형 충돌 판정 알고리즘을 설명하고 구현할 수 있다.

핵심 질문

> 공이 벽과 패들에 '진짜처럼' 튕기게 하려면 무엇을 계산해야 할까?

나의 생각: _____

활동 1. 공 추적표

y=10, vy=-6, size=16, H=500

프레임	y += vy 후	벽 검사	최종 y	vy
1				
2				
3				

활동 2. 충돌 판정 그림

모눈 위에 두 사각형 A(0,0,10,10), B(5,5,10,10)을 그리고 겹친 영역에 색칠하세요.

가로로 겹치는 조건: _____ 세로로 겹치는 조건: _____

핵심 개념 빈칸 채우기

1. 위치 = 위치 + 속도 (_____) 운동)
2. 반사 = 부호 (_____) 위/아래 벽 vy, 패들 vx
3. AABB 충돌: (_____) 모두 겹치면 충돌

4. 충돌 후 (_____) 끼임 버그 방지

자기 평가

항목	잘함	보통	노력 필요
속도 벡터(vx, vy)로 물체의 등속 운동을 구현할 수 있다.	☐	☐	☐
반사의 원리를 속도 부호 변경으로 모델링할 수 있다.	☐	☐	☐
직사각형 충돌 판정 알고리즘을 설명하고 구현할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

17차시 학생 활동지 · Scoreboard와 Game: 점수·승패·재시작으로 완성하기

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 구현 · 게임: 핑퐁 완성본 (main.py · game.py · scoreboard.py)

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 득점과 승패 판정 알고리즘을 구현하고 None을 활용할 수 있다.
- ☐ 글자를 화면 가운데에 정렬하는 좌표를 계산할 수 있다.
- ☐ 여러 클래스가 협력하는 프로그램의 구조(모듈·객체 관계)를 설명할 수 있다.

핵심 질문

> 여러 객체가 각자 맡은 일을 하면서 하나의 게임이 되려면 누가 무엇을 지휘해야 할까?

나의 생각: _____

활동 1. 클래스 구조도 그리기

Game을 가운데에 두고 Game이 가진 객체들과 constants.py의 관계를 화살표로 그리세요.

활동 2. update() 흐름 순서 맞히기

다음을 실행 순서대로 번호를 적으세요.

순서	내용
	공 이동, 벽 튕김
	승자가 있으면 return
	패들 이동
	패들 충돌 검사
	화면 밖이면 득점 + 공 리셋

순서	내용
	승자 문구 설정

핵심 개념 빈칸 채우기

1. (_____) 점수 기억, winner()는 승자 또는 None
2. (_____) 이동 → 충돌 → 득점 → 승패 판정
3. (_____) 정렬 x = 중심 - 너비 // 2
4. (_____) 지휘만, 일은 각 객체가 (역할 분담)

자기 평가

항목	잘 함	보통	노력 필요
득점과 승패 판정 알고리즘을 구현하고 None을 활용할 수 있다.	☐	☐	☐
글자를 화면 가운데에 정렬하는 좌표를 계산할 수 있다.	☐	☐	☐
여러 클래스가 협력하는 프로그램의 구조(모듈·객체 관계)를 설명할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

18차시 학생 활동지 · 테스트 케이스로 오류 찾고 개선하기

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 테스트 및 개선 · 게임: 업다운 개선 버전 (11_updown_safe.py) · 핑퐁 테스트

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 문법 오류·실행 오류·논리 오류를 구분할 수 있다.
- ☐ 정상·경계·예외 입력으로 테스트 케이스를 설계하고 실행 결과를 기록할 수 있다.
- ☐ 입력 검증, try–except, 함수 분리로 오류를 수정하고 회귀 테스트를 할 수 있다.

핵심 질문

> 내가 만든 게임은 친구가 어떤 값을 입력해도 끝까지 올바르게 동작할까?

나의 생각: _____

활동 1. 테스트 케이스 표 (본시 활동지)

번호	구분	입력값	예상 결과	실제 결과	통과(○/×)	수정 내용
1	정상					
2	정상					
3	경계					
4	경계					
5	예외					
6	예외					
7	예외					

활동 2. 교차 테스트 기록 (다른 모둠 게임)

테스트한 모둠	입력/상황	일어난 일	원인 단계 (입력/처리/출력)

활동 3. AI 활용 기록표

차시	활용 목적 (코드 작성/오류 수정)	AI에 입력한 내용 (개인정보 금지)	AI 답변 요약	내가 확인·수정한 내용
18				

핵심 개념 빈칸 채우기

1. 오류 3종: 문법(실행 전) · (_____) · 논리(메시지 없음)
2. 테스트 케이스 = 입력 + 예상 결과, (_____)
3. 검사는 함수 하나에 → 자동 (_____) 가능
4. 고친 (_____) 회귀 테스트

자기 평가

항목	잘함	보통	노력 필요
문법 오류·실행 오류·논리 오류를 구분할 수 있다.	☐	☐	☐
정상·경계·예외 입력으로 테스트 케이스를 설계하고 실행 결과를 기록할 수 있다.	☐	☐	☐
입력 검증, try–except, 함수 분리로 오류를 수정하고 회귀 테스트를 할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

19차시 학생 활동지 · 나만의 게임 프로젝트 기획과 설계

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 공유 및 확장 · 게임: 프로젝트 계획서 · 순서도 · 클래스 구조도

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 배운 게임 중 하나를 골라 개선하거나 새 게임을 기획할 수 있다.
- ☐ 기능 목록을 입력·처리·출력과 하위 문제로 분해하여 계획서를 작성할 수 있다.
- ☐ 순서도·함수 설계표·클래스 구조도(차트)로 프로그램을 설계할 수 있다.

핵심 질문

> 지금까지 배운 6단계를 내 힘으로 다시 밟으려면 무엇부터 계획해야 할까?

나의 생각: _____

활동 1. 프로젝트 계획서

항목	내용
모둠 / 역할	
프로젝트 이름	
문제 정의 (왜 만드나?)	
입력	
처리	
출력	
꼭 필요한 기능 (必)	
있으면 좋은 기능	
사용할 데이터 구조 (리스트/딕셔너리/2차원 리스트)	
일정 (20~22차시)	

활동 2. 함수·클래스 설계표

이름	종류(함수/클래스/메서드)	매개변수	반환값	하는 일

이름	종류(함수/클래스/메서드)	매개변수	반환값	하는 일

활동 3. 설계 차트

① 핵심 기능 순서도

② 클래스 구조도 또는 상태 다이어그램

활동 4. 테스트 계획

번호	구분	입력/상황	예상 결과
1	정상		
2	경계		
3	예외		

핵심 개념 빈칸 채우기

1. 6단계를 모둠이 스스로: 오늘은 (_____)
2. 계획서 = 문제 정의 + IPO + (_____) + 데이터 + 설계 + 테스트 계획
3. 차트: 순서도, 클래스 구조도, 상태 (_____)
4. 설정과 규칙은 (_____) 묶기

자기 평가

항목	잘 함	보통	노력 필요
배운 게임 중 하나를 골라 개선하거나 새 게임을 기획할 수 있다.	☐	☐	☐
기능 목록을 입력·처리·출력과 하위 문제로 분해하여 계획서를 작성할 수 있다.	☐	☐	☐

항목	잘 함	보통	노력 필요
순서도·함수 설계표·클래스 구조도(차트)로 프로그램을 설계할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

20차시 학생 활동지 · 프로젝트 구현: 설계대로 만들고 기능 확장하기

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 공유 및 확장 · 게임: 확장 예제: 공 가속 · AI 패들 · 최고 기록 저장

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 계획서와 설계 차트에 따라 프로그램을 단계적으로 구현할 수 있다.
- ☐ 기존 클래스와 함수를 수정·확장하여 새 기능을 추가할 수 있다.
- ☐ 파일 입출력으로 게임 기록을 저장하고 불러올 수 있다.

핵심 질문

> 설계한 대로 만들려면 어떤 순서로, 얼마나 작게 나누어 구현해야 할까?

나의 생각: _____

활동 1. 구현 진행 기록

순서	기능	구현 완료	테스트 결과	메모 (바뀐 설계 등)
1		☐		
2		☐		
3		☐		
4		☐		

활동 2. AI 활용 기록표

차시	활용 목적	AI에 입력한 내용 (개인정보 금지)	AI 답변 요약	내가 확인·수정한 내용
20				
20				

활동 3. 내 코드 설명하기

내가 작성한 핵심 코드 3줄을 옮겨 쓰고, 각 줄이 하는 일을 설명하세요.

```
```

설명: _____

핵심 개념 빈칸 채우기

1. 작게 만들고 바로 (_____) 점진적 구현
2. 공 가속: (_____) 크기(abs) 분리, 최대 속도 제한
3. AI 패들: 상속 + 데드존 + (_____)
4. 파일 입출력: with open, (_____) 예외 처리

자기 평가

항목	잘함	보통	노력 필요
계획서와 설계 차트에 따라 프로그램을 단계적으로 구현할 수 있다.	☐	☐	☐
기존 클래스와 함수를 수정·확장하여 새 기능을 추가할 수 있다.	☐	☐	☐
파일 입출력으로 게임 기록을 저장하고 불러올 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

21차시 학생 활동지 · 프로젝트 테스트·리팩터링과 차트 작성

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 공유 및 확장 · 게임: 동료 테스트 · 코드 리뷰 체크리스트 · 최종 차트

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 동료 테스트로 발견한 문제를 분석하고 수정할 수 있다.
- ☐ 중복 코드·이름·주석을 점검하여 코드를 리팩터링할 수 있다.
- ☐ 완성한 프로그램과 일치하는 최종 차트(순서도·구조도·테스트 결과표)를 작성할 수 있다.

핵심 질문

> '돌아가는 코드'를 '다른 사람도 이해하고 믿을 수 있는 코드'로 만들려면 무엇을 해야 할까?

나의 생각: _____

활동 1. 버그 리포트 (다른 모둠에게)

번호	상황	입력/행동	기대 결과	실제 결과
1				
2				

활동 2. 리팩터링 체크리스트

점검	우리 코드 상태	고친 내용
중복 코드		
이름		
매직 넘버		
주석		
예외 처리		

활동 3. 최종 차트와 설계 비교

차트	19차시 설계	최종 코드	바뀐 이유
순서도/상태			

차트	19차시 설계	최종 코드	바뀐 이유
클래스·함수 구조			
테스트 통과	/ 개	/ 개	

핵심 개념 빈칸 채우기

1. 버그 리포트: (_____) (재현 가능하게)
2. 리팩터링: (_____) 매직 넘버→상수, 의미 있는 이름, '왜' 주석
3. (_____) 후 회귀 테스트
4. 차트는 완성 코드와 (_____) 한다

자기 평가

항목	잘 함	보통	노력 필요
동료 테스트로 발견한 문제를 분석하고 수정할 수 있다.	☐	☐	☐
중복 코드·이름·주석을 점검하여 코드를 리팩터링할 수 있다.	☐	☐	☐
완성한 프로그램과 일치하는 최종 차트(순서도·구조도·테스트 결과표)를 작성할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____

22차시 학생 활동지 · 게임 시연·발표와 프로젝트 성찰

> 텍스트 게임 프로젝트를 구현하면 배우는 파이썬 프로그래밍
> 단계: 공유 및 확장 · 게임: 갤러리 워크 · 발표 · 동료 평가 · 위키독스/깃허브 공유

학번: _____ 이름: _____ 모둠: _____ 날짜: _____

오늘의 학습 목표

- ☐ 완성한 게임을 시연하고 설계·구현·테스트 과정을 발표할 수 있다.
- ☐ 동료의 게임을 평가 기준에 따라 피드백할 수 있다.
- ☐ 프로젝트 전 과정을 성찰하고 확장 방향을 제안할 수 있다.

핵심 질문

> 우리가 만든 게임과 문제 해결 과정을 다른 사람에게 어떻게 전하고, 다음에는 무엇을 더 할 수 있을까?

나의 생각: _____

활동 1. 동료 평가지

평가한 모둠: _____

평가 항목	★(1~3)	의견
규칙이 쉽고 재미있다		
이상한 입력에도 멈추지 않는다		
함수/클래스로 잘 나뉘고 주석이 있다		
데이터 구조를 알맞게 썼다		
차트와 프로그램이 일치한다		

칭찬 1: _____ 제안 1: _____

활동 2. 프로젝트 성찰지

1. 가장 어려웠던 점과 해결 방법

_____

1. 6단계 중 가장 중요하다고 느낀 단계와 이유

_____

1. 다음에 더 해 보고 싶은 기능

_____

1. 이 수업에서 배운 방법을 쓸 수 있는 다른 문제(다른 교과·생활)

_____

활동 3. 최종 제출 체크리스트

제출물	확인
게임 소스코드 (.py, 주석 포함)	☐
프로젝트 계획서 (19차시)	☐
최종 차트 (순서도/구조도/테스트 결과표)	☐
테스트 케이스 표	☐
AI 활용 기록표	☐
성찰지	☐

핵심 개념 빈칸 채우기

1. 발표 = 문제 정의 + 시연 + 핵심 코드 + 버그 (_____) + 다음 계획
2. 동료 피드백: 칭찬 1 + 제안 1, (_____)
3. 공유: 출처 · (_____) 보호 · AI 활용 공개
4. 6단계 문제 해결 모형을 다른 (_____)

자기 평가

항목	잘함	보통	노력 필요
완성한 게임을 시연하고 설계·구현·테스트 과정을 발표할 수 있다.	☐	☐	☐
동료의 게임을 평가 기준에 따라 피드백할 수 있다.	☐	☐	☐
프로젝트 전 과정을 성찰하고 확장 방향을 제안할 수 있다.	☐	☐	☐

오늘 배운 점 / 궁금한 점

_____

_____